

Índice de contenidos

FUNCIONALIDAD ESENCIAL.....	2
Reindexación.....	2
Eliminar entradas de un eje.....	3
Indexación, selección y filtrado.....	5
Aritmética y alineación de datos.....	8
Aplicación y asignación de funciones.....	11
Ordenación y asignación de rangos.....	12
Índices de ejes con etiquetas duplicadas.....	16
 RESUMIR Y CALCULAR ESTADÍSTICAS DESCRIPTIVAS.....	 17
Métodos de estadística descriptivas y de resumen:.....	18
Métodos únicos, de recuentos de valores y de pertenencia a conjuntos.....	19

FUNCIONALIDAD ESENCIAL

En este apartado veremos la mecánica fundamental de la interacción con los datos contenidos en una serie o dataframe.

Reindexación

Consiste en crear un nuevo objeto con los valores reordenados para que se alineen con el nuevo índice. Se hace llamando a la función **reindex**.

```
import pandas as pd
import numpy as np

# Reindex con Series
obj = pd.Series([4.5, 7.2, -5.3, 3.6], index=["d", "b", "a", "c"])
print(obj)
```

d	4.5
b	7.2
a	-5.3
c	3.6

dtype: float64

```
obj2 = obj.reindex(["a", "b", "c", "d", "e"])
#Vemos que el nuevo índice "e" lo rellena con NaN
print(obj2)
```

a	-5.3
b	7.2
c	3.6
d	4.5
e	NaN

dtype: float64

```
# Reindex con Dataframes
frame = pd.DataFrame(np.arange(9).reshape(3,3),
                      index=["a", "c", "d"],
                      columns=["Ohio", "Texas", "California"])
print(frame)
```

	Ohio	Texas	California
a	0	1	2
c	3	4	5
d	6	7	8

```

states = ["Texas", "Utah", "California"]
frame2 = frame.reindex(states, axis=1) #igual que axis="columns"

print(frame2)

```

	Texas	Utah	California
a	1	NaN	2
c	4	NaN	5
d	7	NaN	8

Argumentos que podemos usar con la función reindex:

Argumento	Descripción
labels	Nueva secuencia a utilizar como índice. Puede ser cualquier estructura de datos de tipo secuencia.
index	Usa la secuencia pasada como nuevas etiquetas de índice
columns	Usa la secuencia pasada como nuevas etiquetas de columna
axis	Determina el eje a reindexar: index (axis=0, por defecto) o columns (axis=1)
method	Para rellenar los valores faltantes cuando se agregan nuevas etiquetas en filas o columnas: "ffill" rellena hacia delante y "bfill" rellena hacia atrás
fill_value	Determina el valor a utilizar para rellenar valores faltantes. El valor predeterminado es "missing" que rellena con valores nulos
limit	Al rellenar valores, determina el número máximo de elementos a rellenar
copy	Si es True, copia los datos subyacentes incluso aunque el nuevo índice sea equivalente al antiguo. Si es False, no copia los datos cuando los índices sean equivalentes

Eliminar entradas de un eje

Se puede hacer de varias formas, empleando la reindexación definiendo un índice que no tenga la entrada que no queremos, o con el método drop.

```

import pandas as pd
import numpy as np

obj = pd.Series(np.arange(5.), index=["a", "b", "c", "d", "e"])
print(obj)

a    0.0
b    1.0
c    2.0
d    3.0
e    4.0
dtype: float64

# Método drop
obj2 = obj.drop(["c", "e"])
print(obj2)

a    0.0
b    1.0
d    3.0
dtype: float64

# Con reindex
nuevo_indice = ["a", "b", "d"]
obj3 = obj.reindex(nuevo_indice)
print(obj3)

a    0.0
b    1.0
d    3.0
dtype: float64

```

Con los dataframes, podemos quitar elementos de cualquier eje.

```

import pandas as pd
import numpy as np

frame = pd.DataFrame(np.arange(16).reshape(4,4),
                      index=["Ohio", "Texas", "California", "Utah"],
                      columns=["Uno", "Dos", "Tres", "Cuatro"])

print(frame)

```

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15

```
frame2 = frame.drop(index="Ohio", columns=["Cuatro", "Dos"])
print(frame2)
```

	Uno	Tres
Texas	4	6
California	8	10
Utah	12	14

Indexación, selección y filtrado

Tanto en las Series como en los Dataframes se pueden emplear corchetes o los operadores **loc** e **iloc** (preferiblemente usar estos dos últimos operadores).

```
import pandas as pd
import numpy as np

# Trabajamos con series
obj = pd.Series(np.arange(4.), index=["a", "b", "c", "d"])
print(obj)

      a    0.0
      b    1.0
      c    2.0
      d    3.0

print(obj[obj < 2])

      a    0.0
      b    1.0

print(obj.loc[["a", "b"]])

      a    0.0
      b    1.0
```

```
#Serie con índice de enteros
```

```
obj2 = pd.Series(np.arange(3.), index=[2, 0, 1])
```

```
print(obj2)
```

```
2    0.0
0    1.0
1    2.0
dtype: float64
```

```
#Observa la diferencia entre iloc y [] con el índice de enteros
```

```
print(obj2.iloc[[0, 1, 2]])
```

```
2    0.0
0    1.0
1    2.0
```

```
print(obj2[[0, 1, 2]]) #trata los enteros como etiquetas
```

```
0    1.0
1    2.0
2    0.0
```

```
# Trabajamos con Dataframes
```

```
frame = pd.DataFrame(np.arange(16).reshape(4,4),
```

```
                    index=["Ohio", "Texas", "California", "Utah"],
```

```
                    columns=["Uno", "Dos", "Tres", "Cuatro"])
```

```
print(frame)
```

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7
California	8	9	10	11
Utah	12	13	14	15

```
print(frame[["Dos", "Tres"]])
```

	Dos	Tres
Ohio	1	2
Texas	5	6
California	9	10
Utah	13	14

```
print(frame > 5)
```

	Uno	Dos	Tres	Cuatro
Ohio	False	False	False	False
Texas	False	False	True	True
California	True	True	True	True
Utah	True	True	True	True

```
print(frame[frame["Uno"] > 5])
```

	Uno	Dos	Tres	Cuatro
California	8	9	10	11
Utah	12	13	14	15

```
print(frame[:2])
```

	Uno	Dos	Tres	Cuatro
Ohio	0	1	2	3
Texas	4	5	6	7

```
print(frame.loc["Texas", ["Uno", "Dos"]])
```

Uno	4
Dos	5

Name: Texas, dtype: int64

```
print(frame.iloc[[1, 2], [3, 0, 1]])
```

	Cuatro	Uno	Dos
Texas	7	4	5
California	11	8	9

```
print(frame.iloc[:, :3][frame.Uno < 7])
```

	Uno	Dos	Tres
Ohio	0	1	2
Texas	4	5	6

[:, :3] todas las filas y las primeras 3 columnas

Opciones de indexado con objetos Dataframe

Tipo	Descripción
df[column]	Selecciona una columna o secuencia de columnas del dataframe
df.loc[rows]	Selecciona una fila o subconjunto de filas por etiqueta
df.loc[:, cols]	Selecciona una sola columna o conjunto de columnas por etiqueta
df.loc[rows, cols]	Selecciona una fila (o filas) y columna (o columnas) por etiqueta
df.iloc[rows]	Selecciona una fila o subconjunto de filas por posición de entero
df.iloc[:, cols]	Selecciona una sola columna o conjunto de columnas por posición de entero
df.iloc[rows, cols]	Selecciona una fila (o filas) y columna (o columnas) por posición de entero
df.at[row, col]	Selecciona un solo valor escalar por etiqueta de fila y columna
df.iat[row, col]	Selecciona un solo valor escalar por posición de fila y columna (enteros)
método reindex	Selecciona filas o columnas por etiquetas

Aritmética y alineación de datos

Pandas simplifica mucho el trabajo con objetos que tienen distinto índice. Por ejemplo, al sumar dos objetos con índices distintos, el resultado incluirá la unión de todos los índices. La alineación interna de datos introduce valores faltantes (NaN) en las ubicaciones de estos casos en los que los índices difieren. Los valores faltantes se propagarán en los cálculos aritméticos posteriores.

Métodos aritmeticos:

Método	Descripción
add, radd	Métodos para suma (+)
sub, rsub	Métodos para resta (-)
div, rdiv	Métodos para división (/). Devuelve el resultado de la división incluyendo la parte fraccionaria
floordiv, rfloordiv	Métodos para división por el suelo o de piso (/). Devuelve el cociente de la división.
mul, rmul	Métodos para multiplicación (*)
pow, rpow	Métodos para exponencial (**)

Cada método tiene un equivalente que comienza con **r** que hace lo mismo pero con los argumentos invertidos, es decir, `obj1.rdiv(obj2)` es lo mismo que `obj2.div(obj1)`.

Puede ser que nos interese rellenar con un valor diferente al NaN cuando se encuentra una etiqueta de un eje en un objeto pero no en el otro. Para ello se emplea el argumento **fill_value**.

```
import pandas as pd
import numpy as np

frame1 = pd.DataFrame(np.arange(12.).reshape(3,4),
                      columns=list("abcd"))
```

	a	b	c	d
0	0.0	1.0	2.0	3.0
1	4.0	5.0	6.0	7.0
2	8.0	9.0	10.0	11.0


```
frame2 = pd.DataFrame(np.arange(20.).reshape(4,5),
                      columns=list("abcde"))
```

	a	b	c	d	e
0	0.0	1.0	2.0	3.0	4.0
1	5.0	6.0	7.0	8.0	9.0
2	10.0	11.0	12.0	13.0	14.0
3	15.0	16.0	17.0	18.0	19.0

```
sumal = frame1 + frame2
print(sumal)
```

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	NaN
1	9.0	11.0	13.0	15.0	NaN
2	18.0	20.0	22.0	24.0	NaN
3	NaN	NaN	NaN	NaN	NaN

```
suma = frame1.add(frame2, fill_value=0)
print(suma)
```

	a	b	c	d	e
0	0.0	2.0	4.0	6.0	4.0
1	9.0	11.0	13.0	15.0	9.0
2	18.0	20.0	22.0	24.0	14.0
3	15.0	16.0	17.0	18.0	19.0

```
division = frame1.div(frame2)
print(division)
```

	a	b	c	d	e
0	NaN	1.000000	1.000000	1.000000	NaN
1	0.8	0.833333	0.857143	0.875000	NaN
2	0.8	0.818182	0.833333	0.846154	NaN
3	NaN	NaN	NaN	NaN	NaN

```
division2 = frame1.rdiv(frame2)
print(division2)
```

	a	b	c	d	e
0	NaN	1.000000	1.000000	1.000000	NaN
1	1.25	1.200000	1.166667	1.142857	NaN
2	1.25	1.222222	1.200000	1.181818	NaN
3	NaN	NaN	NaN	NaN	NaN

También se pueden hacer operaciones entre Series y Dataframes, haciendo coincidir el índice de la serie con las columnas del dataframe (en caso de que coincidan las filas, se debe especificar en el argumento del método aritmético que se use).

```
import pandas as pd
import numpy as np

frame = pd.DataFrame(np.arange(12.).reshape(4,3),
                      columns=list("bde"),
                      index=["Utah", "Texas", "Ohio", "Oregon"])

print(frame)
```

	b	d	e
Utah	0.0	1.0	2.0
Texas	3.0	4.0	5.0
Ohio	6.0	7.0	8.0
Oregon	9.0	10.0	11.0

```
serie = frame.iloc[0]
print(serie)
```

b	0.0
d	1.0
e	2.0

Name: Utah, dtype: float64

```
resta = frame - serie
print(resta)
```

	b	d	e
Utah	0.0	0.0	0.0
Texas	3.0	3.0	3.0
Ohio	6.0	6.0	6.0
Oregon	9.0	9.0	9.0

```
# Ejemplo para ver cómo sería si lo que coinciden son las filas
serie1 = frame["d"]
```

Utah	1.0
Texas	4.0
Ohio	7.0
Oregon	10.0

Name: d, dtype: float64

```
resta2 = frame.sub(serie1, axis=0) #o axis="index"
print(resta2)
```

	b	d	e
Utah	-1.0	0.0	1.0
Texas	-1.0	0.0	1.0
Ohio	-1.0	0.0	1.0
Oregon	-1.0	0.0	1.0

Aplicación y asignación de funciones

Cuando se aplican unfuncs (universal functions) de NumPy a objetos de Pandas, se comportan de manera vectorizada, es decir, operan elemento por elemento sin necesidad de usar bucles explícitos:

```
import pandas as pd
import numpy as np

frame = pd.DataFrame(np.random.standard_normal((4,3)),
                     columns=list("bde"),
                     index=["Utah", "Texas", "Ohio", "Oregon"])

print(frame)
```

	b	d	e
Utah	0.307882	0.413931	-0.001261
Texas	0.285069	0.198186	-0.656558
Ohio	0.376168	-2.194243	-0.639403
Oregon	0.242880	0.522610	0.545270

```
serie = np.abs(frame.iloc[0])
```

b	0.152555
d	0.648521
e	0.176412

Name: Utah, dtype: float64

```
print(np.sqrt(serie))
```

b	0.390583
d	0.805308
e	0.420014

Name: Utah, dtype: float64

También se pueden aplicar funciones a cada columna o fila con el método `apply`.

```
import pandas as pd
import numpy as np

# Calcula la diferencia entre el max y el min de una serie
def f1(x):
    return x.max() - x.min()

frame = pd.DataFrame(np.random.standard_normal((4,3)),
                      columns=list("bde"),
                      index=["Utah", "Texas", "Ohio", "Oregon"])
print(frame)
```

	b	d	e
Utah	0.025398	0.294115	1.508486
Texas	-0.422561	-0.844549	0.765746
Ohio	0.548420	-0.478634	0.226512
Oregon	-0.636527	0.017326	0.944349

```
# La función f1 se invoca una vez para cada columna
print(frame.apply(f1))
```

	b	d	e
	1.184947	1.138664	1.281975

dtype: float64

```
# La función f1 se invoca una vez para cada fila
print(frame.apply(f1, axis="columns"))
```

	Utah	Texas	Ohio	Oregon
	1.483088	1.610295	1.027053	1.580877

dtype: float64

Ordenación y asignación de rangos

Ordenar un conjunto de datos según un cierto criterio es una operación interna importante. Para ordenar lexicográficamente por etiqueta de fila o columna, se emplea el método **sort_index**, que devuelve un objeto nuevo y ordenado. Para ordenar por sus valores se emplea **sort_values**.

```
import pandas as pd
```

```
import numpy as np
frame = pd.DataFrame(np.random.standard_normal((4,4)),
                      columns=list("dabc"),
                      index=["Trenes", "Barcos", "Autobus", "Coche"])
```

	d	a	b	c
Trenes	-0.697110	1.750307	-0.594645	0.685423
Barcos	0.410974	-0.973952	-1.070031	1.308861
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Coche	-0.505677	0.353713	0.625704	2.598641

```
# Ordenamos por filas
print(frame.sort_index())
```

	d	a	b	c
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Coche	-0.505677	0.353713	0.625704	2.598641
Trenes	-0.697110	1.750307	-0.594645	0.685423

```
# Ordenamos por columnas
print(frame.sort_index(axis=1)) # o axis="columns"
```

	a	b	c	d
Trenes	1.750307	-0.594645	0.685423	-0.697110
Barcos	-0.973952	-1.070031	1.308861	0.410974
Autobus	-1.622006	-1.249022	-0.926627	1.039543
Coche	0.353713	0.625704	2.598641	-0.505677

```
# Ordenamos por columnas en orden inverso
print(frame.sort_index(axis=1, ascending=False)) # o axis="columns"
```

	d	c	b	a
Trenes	-0.697110	0.685423	-0.594645	1.750307
Barcos	0.410974	1.308861	-1.070031	-0.973952
Autobus	1.039543	-0.926627	-1.249022	-1.622006
Coche	-0.505677	2.598641	0.625704	0.353713

```
# Ordenamos por valores.
# En caso de empate en "a", ordena por valores en "c"
```

```
print(frame.sort_values(["a", "c"]))
```

	d	a	b	c
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Coche	-0.505677	0.353713	0.625704	2.598641
Trenes	-0.697110	1.750307	-0.594645	0.685423

```
#Si hubiera valores nulos se pueden poner al principio
frame.loc["Trenes", "a"] = np.nan
print(frame.sort_values(["a"], na_position="first"))
```

	d	a	b	c
Trenes	-0.697110	NaN	-0.594645	0.685423
Autobus	1.039543	-1.622006	-1.249022	-0.926627
Barcos	0.410974	-0.973952	-1.070031	1.308861
Coche	-0.505677	0.353713	0.625704	2.598641

La función **rank** de Pandas se utiliza para asignar rangos a los valores de una serie o columna en un DataFrame. Funciona ordenando los valores de menor a mayor y asignando un rango desde 1 hasta n, donde n es el número de valores no nulos. Si existen valores duplicados, la función maneja los empates según el método especificado. Por defecto, usa el método "average", que asigna a los valores repetidos el promedio de los rangos que ocuparían.

```
import pandas as pd

obj = pd.Series([7, -5, 7, 4, 2, 0, 4])

print(obj)
```

0	7
1	-5
2	7
3	4
4	2
5	0
6	4

```
dtype: int64
```

```
print(obj.rank())
```

```
0    6.5
1    1.0
2    6.5
3    4.5
4    3.0
5    2.0
6    4.5
dtype: float64
```

```
# Desempata con el orden en que aparecen los valores
print(obj.rank(method="first"))
```

```
0    6.0
1    1.0
2    7.0
3    4.0
4    3.0
5    2.0
6    5.0
dtype: float64
```

```
# Ejemplo dataframe
frame = pd.DataFrame({"a": [4.3, 7, -3, 2],
                      "b": [0, 1, 2, 0],
                      "c": [-2, 5, 8, -2.5]})
```

```
print(frame)
```

```
   a  b  c
0  4.3  0 -2.0
1  7.0  1  5.0
2 -3.0  2  8.0
3  2.0  0 -2.5
```

```
# Hace el rango por filas
print(frame.rank(axis="columns"))
```

	a	b	c
0	3.0	2.0	1.0
1	3.0	1.0	2.0
2	1.0	2.0	3.0
3	3.0	2.0	1.0

La tabla de a continuación muestra los métodos existentes de desempate con rank.

Método	Descripción
"average"	Método por defecto. Asigna el rango medio a cada entrada de los valores empatados
"min"	Utiliza el rango mínimo para el grupo empatado
"max"	Utiliza el rango máximo para el grupo empatado
"first"	Asigna rangos en el orden en que aparecen los valores en los datos
"dense"	Igual que "min". Pero el rango va aumentando al cambiar entre grupos de repetidos en vez de entre los números repetidos.

Índices de ejes con etiquetas duplicadas

Aunque muchas de las funciones de pandas requieren que las etiquetas de los índices no se repitan entre si, no es algo obligatorio. De hecho, existe la posibilidad de consultar si en un índice hay repetidos o no con la propiedad `is_unique`.

```
import pandas as pd
import numpy as np

obj = pd.Series([7, -5, 7, 4])

      0    7
      1   -5
      2    7
      3    4
      dtype: int64

print(obj.index.is_unique)

      True

frame = pd.DataFrame(np.random.standard_normal((3,3)),
                     index=["Repetido", "Repetido", "Diferente"])
```


	0	1	2
Repetido	0.752429	-0.802467	-0.449333
Repetido	1.311927	-1.583955	0.396837
Diferente	0.514103	-0.713375	0.522409

```
print(frame.index.is_unique)
```

False

RESUMIR Y CALCULAR ESTADÍSTICAS DESCRIPTIVAS

Los objetos de pandas están equipados con un conjunto de métodos matemáticos y estadísticos que permiten analizar y resumir datos fácilmente. La mayoría de estos métodos entran en la categoría de reducciones o estadísticas de resumen, es decir, métodos que toman un conjunto de valores y generan un único resultado (como por ejemplo la suma o el promedio).

Una característica útil es que estos métodos pueden ignorar los valores NaN (datos faltantes) para no afectar los cálculos, en lugar de devolver un resultado NaN. Esto significa que si en una columna hay algunos valores faltantes, pandas puede seguir operando con los datos disponibles sin necesidad de imputarlos antes de calcular estadísticas.

```
import pandas as pd
import numpy as np

frame = pd.DataFrame([[1.4, np.nan], [7.1, -4.5], [np.nan,
np.nan], [0.75, -1.3]],
                      index=list("abcd"),
                      columns=["uno", "dos"])

print(frame)
```

	uno	dos
a	1.40	NaN
b	7.10	-4.5
c	NaN	NaN
d	0.75	-1.3

```
# Suma los valores de las filas de cada columna
print(frame.sum(axis="index"))
```

```

        uno      9.25
        dos     -5.80
        dtype: float64

# No excluimos los valores faltantes
print(frame.sum(axis="index", skipna=False))

        uno      NaN
        dos      NaN
        dtype: float64

```

Métodos de estadística descriptivas y de resumen:

Método	Descripción
count	Número de valores que no son nulos
describe	Calcula un conjunto de estadísticas de resumen: count, mean, std, min, max y percentiles
min, max	Calcula los valores mínimo o máximo, respectivamente
argmin, argmax	Devuelve la posición del valor mínimo o máximo, respectivamente
idxmin, idxmax	Devuelve los nombres de los índices en las que están el valor mínimo o el máximo, respectivamente
quantile	Calcula el cuantil de muestra entre 0 y 1. Por defecto calcula el 0.5
sum	Suma de valores
mean	Calcula la media
median	Calcula la mediana, es decir, el cuantil 50%
mad	Desviación media absoluta del valor promedio
prod	Producto de todos los valores
var	Varianza de los valores de la muestra
std	Desviación estándar entre los valores de la muestra
skew	Asimetría (tercer momento) de los valores de la muestra. Determina qué tan simétrica es la distribución, si es positiva la cola está hacia la derecha y si es negativa, hacia la izquierda.

kurt	Curtosis (cuarto momento) de los valores de la muestra. Mide qué tan concentrados están los valores en los extremos de una distribución.
cumsum	Suma acumulada de los valores
cummin, cummax	Mínimo o máximo acumulado de los valores, respectivamente
cumprod	Producto acumulado de valores
diff	Calcula la primera diferencia aritmética. Útil para analizar tendencias
pct_change	Calcula cambios de porcentaje
cov	Calcula la covarianza, cómo varían dos variables juntas
corr	Calcula la correlación, que es la covarianza normalizada

Métodos únicos, de recuentos de valores y de pertenencia a conjuntos

Método	Descripción
isin	Devuelve un array booleano indicando si cada valor de una serie o dataframe está contenido en la secuencia de valores que se le pasa como argumento
get_indexer	Encuentra las posiciones de etiquetas en un índice y devuelve sus ubicaciones numéricas en forma de array, útil para alineación de datos y búsqueda eficiente
unique	Devuelve un array de valores únicos de una serie, manteniendo el orden en que aparecen
value_counts	Cuenta la frecuencia de cada valor único en una Serie y devuelve un resultado ordenado de mayor a menor frecuencia.

```
import pandas as pd

serie = pd.Series([3, 1, 2, 3, 2, 4, 1, 3])

conteo = serie.value_counts()
print(conteo)
```

```
3    3
1    2
2    2
4    1
Name: count, dtype: int64
```

```
print(serie.isin([2]))
```

```
0    False
1    False
2     True
3    False
4     True
5    False
6    False
7    False
dtype: bool
```

```
print(serie.unique())
```

```
[3 1 2 4]
```

```
print(conteo.index.get_indexer([3,4]))
```

```
[0 3]
```